Guion de la presentación final (10 minutos, con demo en vivo)

El manual pide una presentación de 10 minutos con **demostración en vivo del autoescalado**. El clúster de AWS se destruyó al terminar (cuesta ~USD 8 al día encendido): para la demo se recrea con `terraform apply` (~15 min) antes de la clase y se destruye después. Una diapositiva por bloque.

Min	Bloque	Qué se dice / qué se enseña
0:00–0:45	**Problema**	Un juego en línea tiene picos: 100 jugadores a las 3 p. m., 1,000 a las 9 p. m. Pagar servidores para el pico todo el día es caro; quedarse corto tira partidas. Objetivo: que la plataforma crezca y se encoja sola, y medir cuánto aguanta y cuánto cuesta.
0:45–1:30	**El juego**	10 s de partida (5 vs 5, servidor autoritativo a 20 ticks/s). "El juego es el caso de estudio; lo que se evalúa es la infraestructura."
1:30–3:00	**Arquitectura**	Diagrama del reporte (sección 2): matchmaker → Agones → GameServers; Prometheus/Grafana; Lambda + API Gateway + DynamoDB; CI/CD a ECR; Terraform. Las **tres capas de escalado**: flota (FleetAutoscaler), pods (HPA del matchmaker), nodos (Cluster Autoscaler).
3:00–6:00	**Demo en vivo del autoescalado**	Ver abajo.
6:00–7:30	**Resultados**	Curva CCU vs latencia p95, servidores por estado, nodos. Rampa de 300 en EKS: flota 3→37 servidores, nodos 4→6, CCU máximo 252, ~42 jugadores por nodo, primer escalado de la flota a los 43 s, ~1 min por nodo. Gráficas: `entrega/evidencias/3.4-autoescalado/reaccion_autoescalado.png` y la captura de Grafana.
7:30–8:30	**Costos**	Clúster USD 0.34/h en reposo y 0.74/h en el pico, **USD 0.003 por jugador-hora**; bots USD 0.60/h para 2,000 (3.1); 20,000 espectadores = 54 TB/h de egress; tabla de líderes ≈ USD 6–7/mes serverless vs ≈ 15–20 una VM.
8:30–9:30	**Lo que aprendimos**	Fallas reales que encontraron las pruebas: `/ready` una sola vez, `SIGTERM` ignorado, EKS sin metrics-server, el `scan` del top con la cuota de 10 Lambdas, y el orden del `destroy` (borraba la ruta a internet antes que Agones). La GPU fue 68 % más lenta que la CPU para un bot pequeño. Recomendaciones de `bufferSize`, tipo de máquina y nodos mínimos.
9:30–10:00	**Cierre**	`terraform destroy` borra todo; "todo lo que vieron se recrea con un comando". Preguntas.

Demo en vivo (3 minutos)

Preparación antes de clase (no en vivo): clúster arriba, flota con buffer, Grafana abierto en el tablero, Locust listo (mejor en una VM de AWS que en la laptop: arriba de 200 jugadores la laptop se satura).

1. **Grafana en pantalla completa** con los paneles CCU, servidores por estado y nodos.
2. Terminal visible: `bash kubectl get gameservers kubectl get nodes`

3. Arrancar la carga (pico corto para que se vea en 3 min): `bash locust -f locustfile.py Jugador -u 300 -r 15 -t 3m --headless --processes -1`
4. Narrar mientras sube: "el CCU sube → los servidores `Ready` pasan a `Allocated` → el FleetAutoscaler crea más para mantener el 20 % libre → cuando no caben, los pods quedan pendientes y el Cluster Autoscaler agrega un nodo".
5. Repetir `kubectl get gameservers` y `kubectl get nodes` para enseñar los nuevos.
6. Si un nodo nuevo tarda más de lo que dura la demo: tener lista la captura de la rampa de 300 (`02-grafana-rampa-300.png`) como respaldo y decirlo.

Ensayo sin nube: igual, pero con kind y la rampa local de 120 jugadores; ahí solo escalan flota y pods (kind tiene un nodo fijo). Sirve para practicar la narración, no como evidencia.

Diapositivas sugeridas (9)

1. Título y equipo
2. Problema
3. El juego (captura)
4. Arquitectura (diagrama)
5. Tres capas de escalado
6. Demo (solo el título; se cambia a Grafana)
7. Resultados (2 gráficas)
8. Costos (tabla)
9. Hallazgos y recomendaciones